JOHANNES KEPLER
UNIVERSITY LINZ

Sequential Elements and Static Timing Analysis



INSTITUTE FOR
INTEGRATED CIRCUITS &
QUANTUM COMPUTING

Simon Dorrer

KV VLSI Design, 336.051, WiSe 26/27

JOHANNES KEPLER
UNIVERSITÄT LINZ
Altenberger Straße 69
4040 Linz, Österreich
jku.at

Today's Goals

After this lecture, you can

- explain **latches and flip-flops**, and why we use flip-flops
- name the **timing parameters** of a flip-flop
- write the **setup and hold conditions** with skew and uncertainty
- explain what **static timing analysis** checks, and what not
- write **SDC constraints**: clocks, environment, exceptions
- read **OpenSTA** reports, and fix setup and hold violations
- explain **time borrowing** in latches



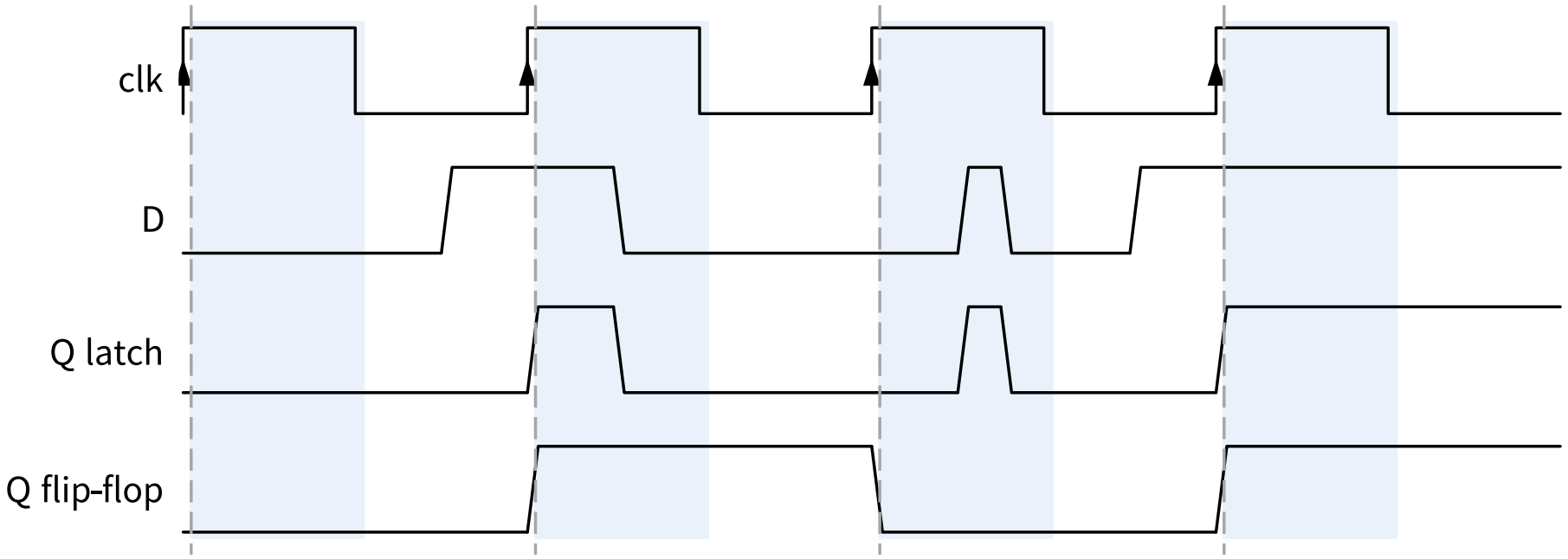
Chapter 6

Lecture notes: Chapter 6 of iic-jku.github.io/vlsi-design

Latches and Flip-Flops



Latch or Flip-Flop?

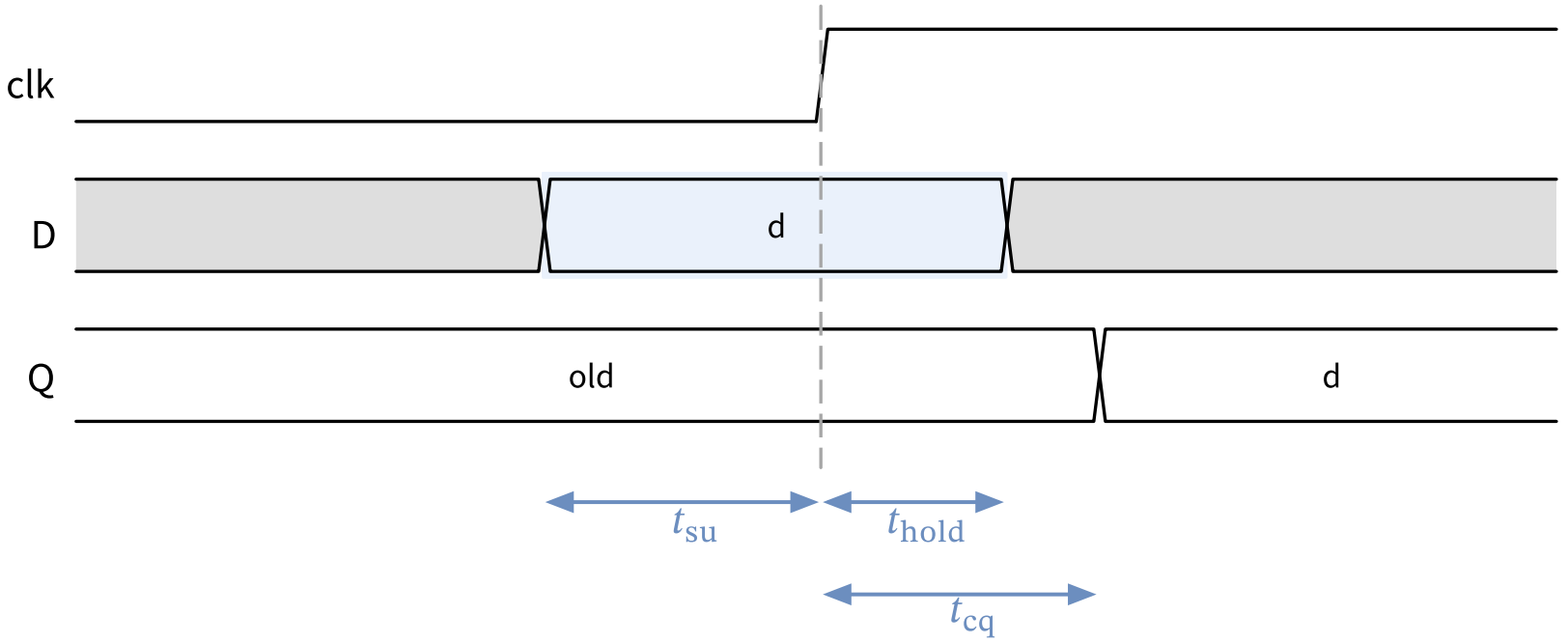


- **Latch**: transparent while the gate is active, `dlhq_1`: 30.84 µm²
- **Flip-flop**: master and slave latch, samples at the edge, `dfrbpq_1` with reset: 48.99 µm²

Timing Parameters of a Flip-Flop



Setup, Hold, Clock-to-Output



- D must be stable from **setup** before to **hold** after the edge
- Q changes **clock-to-output** delay after the edge, a table over clock slew and load

dfrbpq_1 in Numbers

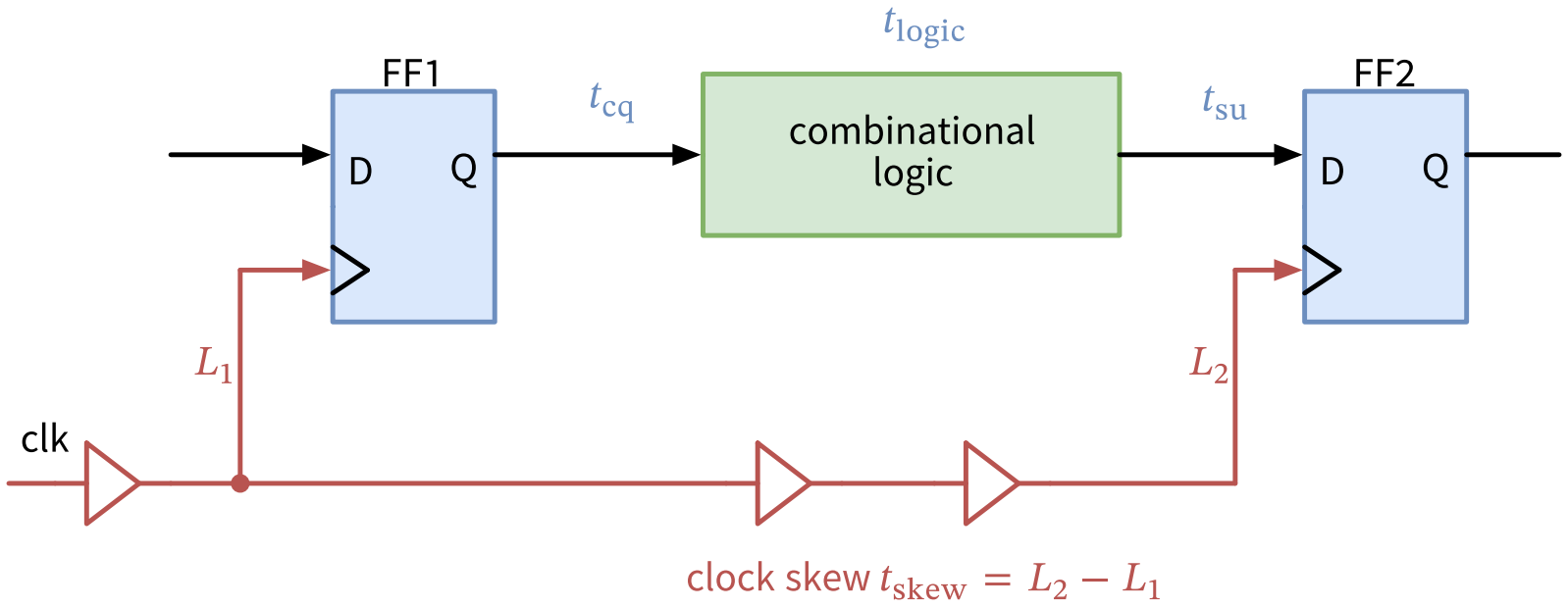
Parameter	Value
clock-to-output, Q rising / falling	0.207 / 0.203 ns
setup, D rising / falling	0.127 / 0.121 ns
hold, D rising / falling	−0.072 / −0.043 ns
recovery / removal of RESET_B	0.133 / −0.114 ns
minimum pulse width, CLK high / low	0.064 / 0.104 ns

- Typical corner, edges of 0.1 ns, 5 fF load, `ff/ff.tcl` with `report_dcalc`
- **Negative hold**: `D` may change shortly before the edge
- **Recovery and removal**: setup and hold of the reset release
- Window violated: **metastable**, Chapter 8

Register-to-Register Paths



Setup and Hold Conditions



- **Setup**: $t_{cq} + t_{logic,max} + t_{su} \leq T + t_{skew} - t_{unc}$
- **Hold**: $t_{cq} + t_{logic,min} \geq t_{hold} + t_{skew} + t_{unc}$

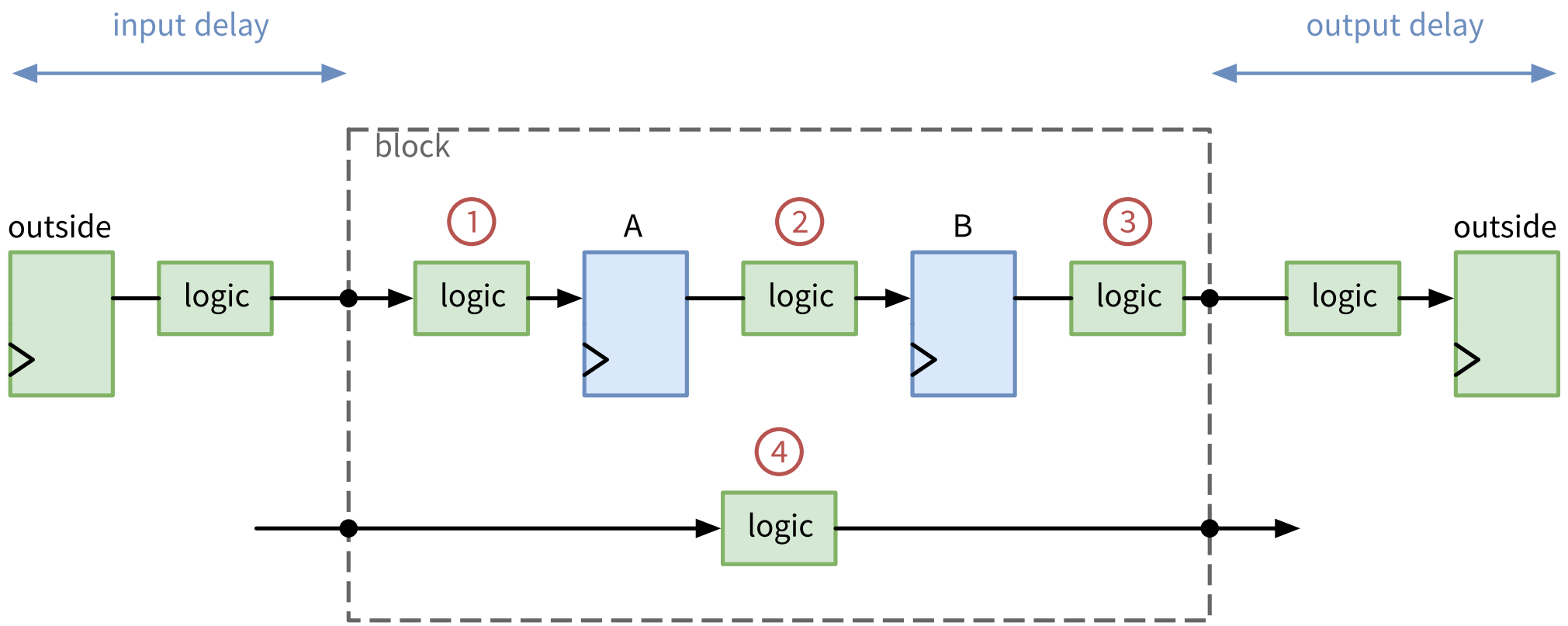
Three Consequences

- Setup sets the **clock frequency**: $f_{max} = 1 / T_{min}$
  - 0.207 + 1 + 0.127 = 1.334 ns for 1 ns of logic, no skew, no uncertainty
- Hold does **not contain T**: a slower clock does **not** fix a hold violation
- A **late capture clock** helps setup and hurts hold by the same amount
  - short paths with late capture clocks: shift registers, scan chains

Static Timing Analysis



Four Kinds of Timing Paths



- Start: input or clock pin. End: output or data pin of a register

Arrival, Required, Slack

- **Arrival times** propagate through the timing graph, max for setup, min for hold
  - each cell: delay from its tables, with input slew and output load
- **Required time** from the capturing edge and the setup or hold time
- **Slack**: required − arrival (setup), arrival − required (hold), ≥ 0 is met
- **WNS**: worst slack, **TNS**: sum of all negative slacks
- No stimuli, every path, but no function and only **what the constraints say**
  - also times paths that can never be activated
- Setup usually hardest in the **slow**, hold in the **fast** corner, sign-off: **all**

Timing Constraints



SDC for bs_reg

```
create_clock -name clk -period 4.0 [get_ports clk_i]
set_clock_transition 0.1 [get_clocks clk]
set_clock_uncertainty 0.1 [get_clocks clk]
set inputs [delete_from_list [all_inputs] [get_ports clk_i]]
set_input_delay 1.0 -clock clk $inputs
set_output_delay 1.0 -clock clk [all_outputs]
set_driving_cell -lib_cell sg13cmos5l_buf_2 -pin X $inputs
set_load 0.01 [all_outputs]
```

- **Ideal clock** until the clock tree exists, then `set_propagated_clock`
- **Input delay**: what the outside path to an input already uses
- **Output delay**: what the outside path from an output needs
- **Driving cell** and **load**: input slew and driver delay, output capacitance

Input and Output Delays So Far

Chapter	Input and output delay	Meaning
3: excl_cnt	0	inputs change at the edge, outputs sampled one edge later
4: FIFO	half a period	testbench drives and samples at the falling edge
6: bs_reg	1 ns of 4 ns	external registers use part of the period

Static Timing Analysis with OpenSTA



The Timing Script

```
# $lib, $corner: set at the top of the file
read_liberty $lib/lib/sg13cmos5l_stdcell_$corner.lib
read_verilog bs_reg.nl.v
link_design bs_reg
read_sdc syn/bs_reg.sdc
check_setup -verbose
report_checks -path_delay max -fields {fanout} -digits 3
report_checks -path_delay min -digits 3
report_clock_min_period
report_wns -digits 3
report_tns -digits 3
report_check_types -max_slew \
  -max_capacitance -violators
```

- `check_setup` **first**: missing clocks and constraints
- Worst path per **path group**: each clock, and `asynchronous`
- `CORNER=slow_1p08V_125C` for the slow corner

Reading a Timing Report

```
  0.000    0.000   clock clk (rise edge)
  0.000    0.000   clock network delay (ideal)
  0.204    0.204 ^ _1719_/Q (dfrbpq_1)
  0.111    0.314 ^ _0812_/X (buf_4)
                 ... three more buffers ...
  0.276    1.147 v _0984_/X (mux4_1)
  0.132    1.279 v _1249_/X (mux2_1)
  0.154    1.433 ^ _1261_/Y (a221oi_1)
  0.059    1.491 v _1262_/Y (o21ai_1)
           1.491   data arrival time
  4.000    4.000   clock clk (rise edge)
 -0.100    3.900   clock uncertainty
 -0.100    3.800   library setup time
           3.800   data required time
           2.308   slack (MET)
```

- Upper half: **arrival**
- Lower half: **required**
- From `shamt_q[1]` to `out_q[30]`
- Four buffers, each drives 9 or 10 pins
- Between registers: **1.69 ns**, 591 MHz

Hold and Reset Checks

- Worst hold: `data_q[0]` to `out_q[16]`
  - arrival 0.328 ns, required 0.100 + (−0.019) = 0.081 ns, slack 0.247 ns
- Group `asynchronous`: recovery and removal of `RESET_B`
  - `rst_ni` drives **71** flip-flops: 0.599 ns, slew of 0.87 ns
  - recovery time grows to 0.323 ns, slack 1.977 ns
- With 1 ns input delay, the reset limits the block: **2.02 ns**, 494 MHz
  - `report_clock_min_period -include_port_paths`
- The reset net: buffered in the layout, released synchronously (Chapter 8)

Live Demo: STA of bs_reg

```
LIB=$PDK_ROOT/ihp-sg13cmos5l/libs.ref/sg13cmos5l_stdcell/lib
LIB=$LIB/sg13cmos5l_stdcell_typ_1p20V_25C.lib
cd content/sequential/code/bs_reg
yosys -p "read_verilog -sv \
  ../../../verification/code/barrel_shift/rtl/barrel_shift.sv \
  rtl/bs_reg.sv; synth -top bs_reg; flatten; dfflibmap -liberty $LIB; \
  abc -constr ../../../synthesis_lib/code/synth/abc.constr -liberty $LIB; \
  opt_clean; stat -liberty $LIB; write_verilog -noattr bs_reg.nl.v"
sta -no_init -exit syn/sta.tcl
```

- 915 cells, `check_setup` clean, setup slack 2.308 ns, hold slack 0.247 ns

Exercise (Section 6.7): how fast is the barrel shifter with the default mapping, in two corners?

Generated Clocks



A Clock from a Register

- Clocks propagate through buffers and gates, **not through a flip-flop**
- A register output as a clock: STA does not know it unless the SDC defines it

```
create_generated_clock -name clk_div2 -source [get_ports clk_i] \
  -divide_by 2 [get_pins div_reg/Q]
```

Exercise (Section 6.8): a clock that STA does not see, in the LED blinker of Chapter 3.

Timing Exceptions



False Paths and Multicycle Paths

- `set_false_path`: never timed, e.g., a static configuration register
- `set_multicycle_path N -setup`: N periods, **plus** `N-1 -hold`

bs_reg at 1.2 ns	Setup slack	Hold slack
no exception	−0.492 ns	0.247 ns
-setup 2	0.708 ns	−0.953 ns
-setup 2, -hold 1	0.708 ns	0.247 ns

- Launch at 0, setup capture at 2T: the hold edge moves to T, `-hold 1` moves it back to 0

Hold Violations



Fixing Hold

- After clock tree synthesis: `set_propagated_clock`, **real skew**
- The layout flow inserts **delay cells** into short paths
  - `dlygate4sd1_1` to `dlygate4sd3_1`, increasing delay
- Setup: shorter path, stronger cells, pipelining, slower clock
- Hold: longer shortest path or less skew, without breaking setup

Exercise (Section 6.10): find and fix a hold violation in a shift register, in three corners.

Latches and Time Borrowing



A Latch Borrows Time

- Latch transparent while the clock is **low**, in the middle of the path
- Late data passes at once: the stage **borrows** from the next one

Middle element	Setup slack, stage 1	Stage 2
flip-flop, falling edge	−0.474 ns	1.001 ns
latch	0 (borrows 0.354 ns)	0.620 ns

- Most to borrow: transparent phase minus setup of the latch
- Price: complex analysis, harder hold checks, scan test

Summary

- Flip-flop: master and slave latch, **setup, hold, clock-to-output**
- Setup sets the **frequency**, a slower clock does **not** fix hold
- A late capture clock helps setup and hurts hold, uncertainty hurts both
- **STA**: all paths, no stimuli, only as good as the **constraints**
- SDC: clocks, generated clocks, input and output delays, exceptions
- Hold fixes: **delay cells**, checked in **all corners**
- Latches **borrow time**, at a price

Exercises

1. How Fast Is the Barrel Shifter? (6.7)
2. A Clock That STA Does Not See (6.8)
3. Fix a Hold Violation (6.10)

All code in `content/sequential/code` of the course repository, every number reproduced by `scripts/check_sequential.sh`



Chapter 6



JOHANNES KEPLER
UNIVERSITY LINZ